

MVP PRD — School Management & Academic Intelligence SaaS

Version: V1.0

Product Type: Multi-tenant SaaS

Target: Private schools

Initial Scale: ~5,000 students across multiple schools

Frontend: React

Backend: NestJS

Database: PostgreSQL

1. Product Objective

Build a multi-tenant school management platform that allows schools to digitally manage:

- Students
- Teachers
- Classes and subjects
- Exams/tests
- Student results
- Fees
- Academic performance

The key differentiator is **academic performance intelligence**, allowing teachers, class incharges, admins, and principals to understand performance rather than simply storing marks.

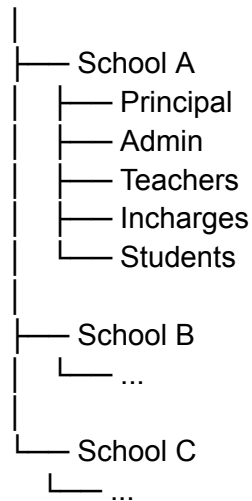
2. User Roles

V1 has five roles:

1. **Student**
2. **Teacher**
3. **Class Incharge**
4. **Admin**
5. **Principal**

Each user belongs to exactly one school organization.

SaaS



3. Multi-Tenancy

The system must support multiple schools from day one.

Every school is an **organization/tenant**.

Requirements

- Each organization has isolated data.
- Users belong to an organization.
- Users can only access their organization's data.
- Backend must derive the organization from authenticated user context.
- Frontend must never be trusted to provide the organization ID.
- PostgreSQL Row-Level Security should be considered as an additional isolation layer.

Example

School A → organization_id = 1

School B → organization_id = 2

A School A user must never be able to access School B's:

- Students
 - Teachers
 - Results
 - Classes
 - Fees
 - Analytics
-

4. Organization Management

The system needs a basic organization structure.

School

Fields:

- School ID
- School name
- Logo
- Address
- Phone
- Email
- Academic year
- Current term
- Status

Academic Year

Example:

2026–2027

Terms

Example:

Term 1

Term 2

Term 3

Exam/Test Types

V1 should support:

- Tests
- Monthly Tests
- Exams
- Term Exams

5. Student Management

Admin can:

- Create student
- Edit student
- View student
- Deactivate student
- Assign student to class
- Assign student to section
- View student history

Student information

Student

- ID
- Name
- Father/Guardian Name
- Date of Birth
- Gender
- Admission Number
- Contact Information
- Class
- Section
- Admission Date
- Status

Student should maintain historical academic records.

6. Teacher Management

Admin can:

- Create teacher
- Edit teacher
- Activate/deactivate teacher
- Assign teacher to subject
- Assign teacher to class
- Assign teacher as class incharge

Teacher assignments

Example:

Teacher: Ahmed

Class: 8-A

Subject: Mathematics

A teacher should only access the academic data relevant to their assignments.

7. Class & Section Management

Admin can:

- Create class
- Create section
- Edit class
- Assign students
- Assign teachers
- Assign class incharge

Example:

Class 8

- └─ Section A
- └─ Section B
- └─ Section C

Each section should be treated as an independent class group for analytics.

8. Subject Management

Admin can:

- Create subjects
- Assign subjects to classes
- Assign teachers to subjects

Example:

Class 8-A

Mathematics → Teacher A
English → Teacher B
Physics → Teacher C
Chemistry → Teacher D

9. Examination Management

Admin can create:

- Test
- Monthly Test
- Exam
- Term Exam

Each examination should contain:

- Name
- Type
- Academic year
- Term
- Class
- Section
- Subject
- Maximum marks
- Passing marks
- Exam date
- Status

Exam states

Draft

↓

Open

↓

Marks Entry

↓

Completed

↓

Published

Only **published** results should become visible to students.

10. Marks Management

Teachers can enter marks for their assigned subjects/classes.

Methods

V1 should support:

Option 1 — Manual entry

Teacher enters marks directly into a table.

Option 2 — Excel import

Teacher/admin uploads an Excel file containing marks.

Excel import is highly valuable because schools may already maintain marks in spreadsheets.

Validation

System should detect:

- Invalid student
 - Duplicate marks
 - Marks greater than maximum marks
 - Missing marks
 - Incorrect subject/class
 - Invalid student-class mapping
-

11. Result Management

After marks are entered, system calculates:

- Obtained marks
- Percentage
- Grade
- Pass/fail
- Subject average
- Class average

Depending on school configuration.

Student result

Mathematics	72/100	72%
English	81/100	81%
Physics	65/100	65%
Chemistry	77/100	77%

Overall	295/400	73.75%
---------	---------	--------

12. Student Performance Analytics

The system should compare results across assessments.

For example:

Term 1 average: 61%

Term 2 average: 68%

Improvement: +7%

Identify:

Strong students

Students performing consistently above a configurable threshold.

Weak students

Students performing below a configurable threshold.

Most improved

Largest positive change between comparable assessments.

Declining students

Significant negative change between assessments.

Weak subjects

Subjects where the student's performance is consistently low.

13. Student Portal

Students can view:

Dashboard

- Current class
- Latest result
- Overall percentage
- Performance trend
- Fee status

Results

- Tests
- Monthly tests

- Exams
- Term exams
- Subject-wise marks
- Grades
- Percentage

Performance

- Overall trend
- Subject performance
- Strong subjects
- Weak subjects
- Improvement
- Decline

Fees

- Current fee
- Paid/unpaid
- Fee amount
- Fee history
- Challan information

Students **cannot edit academic or fee information.**

14. Teacher Portal

Teacher dashboard contains:

Assigned classes

8-A → Mathematics

8-B → Mathematics

9-A → Mathematics

Performance

For each assigned class:

- Class average
- Pass percentage
- Subject average
- Strong students
- Weak students
- Best students

- Most improved
- Declining students
- Previous vs current performance

Teacher can:

- Enter marks
- Upload marks
- View results
- View analytics for assigned subjects/classes

Teacher cannot access unrelated subjects/classes.

15. Class Incharge Portal

The class incharge gets a **whole-class view**.

Example:

Class 8-A

Overall Average: 68%

Mathematics	61%
English	72%
Physics	69%
Chemistry	74%

Analytics

- Overall class average
- Subject averages
- Pass percentage
- Strong students
- Weak students
- Best students
- Most improved
- Declining students
- Subject performance
- Previous vs current assessment

The incharge can see all subjects for their assigned class.

16. Admin Portal

Admin has operational control.

Student management

- Add student
- Edit student
- Assign class
- View student
- Manage status

Teacher management

- Add teacher
- Edit teacher
- Assign subjects
- Assign classes
- Assign incharge

Academic management

- Classes
- Sections
- Subjects
- Academic years
- Terms
- Tests
- Exams
- Marks
- Results

Fee management

- Fee structure
- Generate challan
- Record fee
- Update fee status
- Paid/unpaid
- Fee history
- Outstanding fees

Analytics

Admin can see school-wide:

- Student performance

- Class performance
 - Subject performance
 - Teacher/class performance
 - Weak students
 - Strong students
 - Most improved students
-

17. Principal Portal

The principal gets a **high-level school intelligence dashboard**.

School overview

Total Students 1,250
Overall Average 68%
Pass Rate 87%
Improving Students 642
Declining Students 118

Class comparison

Class	Previous	Current	Change
8-A	61%	68%	+7%
8-B	67%	64%	-3%
9-A	72%	76%	+4%

Teacher performance

Teacher performance should be based on **observable academic metrics**, not a simplistic arbitrary score.

Example:

Teacher: Ahmed
Subject: Mathematics
Class: 8-A

Previous Average: 61%
Current Average: 68%
Change: +7%

Students improved: 24
Students declined: 4
Pass rate: 91%

Principal can identify

- Classes improving
 - Classes declining
 - Weak subjects
 - Strong subjects
 - Weak students
 - Strong students
 - Most improved students
 - Students with declining performance
 - Teacher/class performance trends
-

18. Fee Management

V1 includes basic fee management.

Admin can:

- Define fee amount
- Create fee record
- Generate challan
- Set due date
- Mark paid
- Mark unpaid
- Record payment date
- View payment history
- View outstanding fees

Student can:

- View current fee
- View paid/unpaid status
- View fee history
- View challan

No automated fee notifications in V1.

19. Dashboard & Analytics Requirements

Dashboards should prioritize **actionable information**, not just charts.

Bad:

Average = 68%

Better:

Average = 68%, up 7% from previous term.

Best:

Class 8-A improved 7%, but Mathematics declined 4%. 11 students are below the passing threshold.

The analytics layer should answer:

1. What is happening?
 2. Where is the problem?
 3. Which students need attention?
 4. Has performance improved?
 5. Which subjects/classes are changing?
-

20. Authentication & Authorization

V1 requires:

- Login
- Logout
- Password reset
- Session/token management
- Role-based access control
- Organization-level access control

Roles:

STUDENT
TEACHER
INCHARGE
ADMIN
PRINCIPAL

Every API endpoint must enforce authorization.

21. Audit Logs

Because this is a school system, important administrative actions should be recorded.

Examples:

Admin created student
Admin changed fee
Teacher entered marks
Teacher updated marks
Admin published result
Admin assigned teacher

Audit log:

User
Action
Resource
Timestamp
Old value
New value

This is particularly important for marks and fee changes.

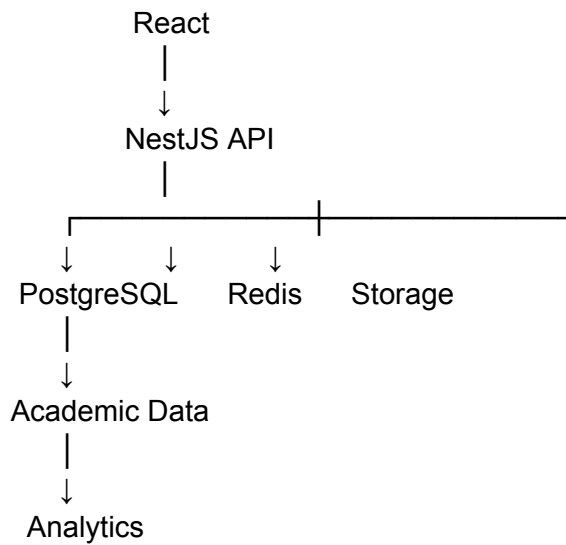
22. V1 Non-Goals

Explicitly do NOT build these in V1:

-  Parent portal
-  WhatsApp notifications
-  SMS
-  Email automation
-  Push notifications
-  Attendance
-  Homework
-  Timetable
-  Payroll
-  HR
-  Transport
-  Library
-  Inventory
-  AI chatbot
-  AI teacher evaluation
-  Microservices

These can be considered after the core system is validated.

23. Recommended V1 Architecture



Initially:

React → NestJS → PostgreSQL

Redis can be introduced when actual usage demonstrates a need.

24. Core Database Entities

At minimum:

Organization
User
Role
Student
Teacher
Class
Section
Subject
TeacherAssignment
StudentEnrollment
AcademicYear
Term
Assessment
AssessmentSubject
Mark
Result
FeeStructure

Fee
Payment
AuditLog

The most important design requirement is that tenant-owned entities are associated with an **Organization**.

25. V1 Success Criteria

The MVP should be considered successful when a real school can perform this complete workflow:

